

UNIDAD 5



Programación Procedural Arreglos Dinámicos

Arreglos dinámicos en Lenguaje C

Un arreglo de 10 componentes enteras se puede declarar de dos maneras:

int b[10] ó int *b.

Arreglo Estático

```
void carga (int x[ ],int n)  
{  
    for(int i=0; i<n; i++)  
        scanf("%d",&x[i]);  
}
```

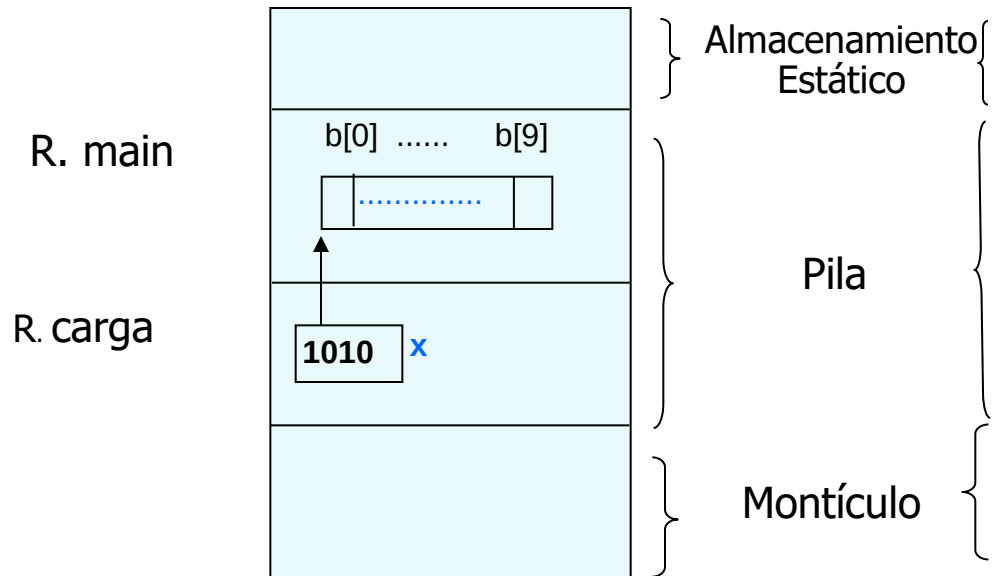
```
int main(void)  
{  
    int b[10];  
    carga(b,10);  
}
```

Arreglo Dinámico

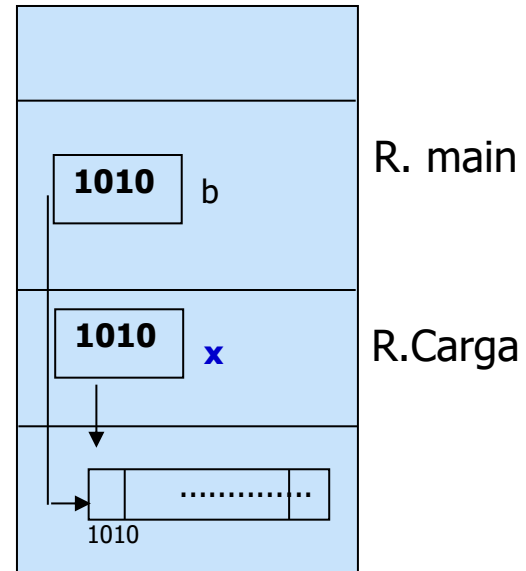
```
void carga ( int *x, int n)  
{  
    for (int i=0; i<n; i++)  
        scanf("%d",&x[i]);  
}
```

```
int main(void)  
{ int *b;  
  b=(int *) malloc (sizeof(int)*10);  
  carga(b,10);  
  free(b);  
}
```

Arreglo Estático



Arreglo Dinámico



Actividad

¿Qué modificaciones deberá realizar si la solicitud de espacio de memoria para almacenar el arreglo, se realiza dentro de la función carga? Realice el mapa de memoria.

Arreglo Dinámico

```
void carga( int *&x)
{
    x= (int*) malloc ( sizeof (int) *10);

    for(int i=0; i<10;i++)
        scanf("%d ",&x[i]);
}

void main(void)
{
    int *b;
    carga(b);
    free(b);
}
```

EJEMPLO

El Instituto Provincial de la Vivienda ha implementado un sistema que consta de 5 planes de pago distintos, con el fin de que los adjudicatarios de sus viviendas puedan cancelar sus deudas. Por cada uno de los 5 planes, se ingresa en forma ordenada la cantidad de adjudicatarios adheridos y por cada uno de ellos el monto adeudado.

Se necesita realizar un programa, que utilizando de manera óptima funciones, informe:

- Para cada plan:
 - Monto total adeudado por los adjudicatarios adheridos al mismo.
 - Monto promedio adeudado.
 - Cantidad de usuarios cuyo monto adeudado es superior al promedio calculado.
- El o los números de planes con mayor cantidad de adjudicatarios

```
#include <malloc.h>
#include <conio.h>
#include <stdio.h>
```

```
void carga( float *t, int xcu)
```

```
{ for( int i=0; i<xcu; i++)
  { printf("\n deuda usuario %3d ",i+1 );
    scanf("%f",&t[i]);
  }
}
```

```
float total( float *t, int cu)
```

```
{ float prom=0;
  for(int i=0; i<cu; i++)
    prom=prom+ t[i];
  printf("\n Tot.adeudado %5.2f ", prom );
  return prom/cu;
}
```

```
int cantidad( float *t, int cu, float prom)
```

```
{ int c=0;
  for( int i=0; i<cu; i++)
    if (t[i]>prom)
      c++;
  return(c);
}
```

```
void maximo(int *tot,int N)
```

```
{ int max=0;
  for( int i=0; i<N;i++)
    if (tot[i]>max)
      max=tot[i];
  printf("\n Planes con mayor cantidad de adj." );
  for (int i=0; i<N; i++)
    if (tot[i] ==max )
      printf("\n %3d ",i+1 );
  return;
}
```

```
{int i,cu,totu[5];
```

```
float *monto, prom;
```

```
for(i=0; i<5;i++)
```

```
{ scanf("%d",&cu);
```

```
tot[i]=cu;
```

```
monto=(float *)malloc(cu * sizeof(float));
```

```
carga(monto, cu);
```

```
prom= total(monto, cu);
```

```
printf("\n Promedio adeudado ", i+1, prom );
```

```
printf("Adjudicatarios con monto superior a
```

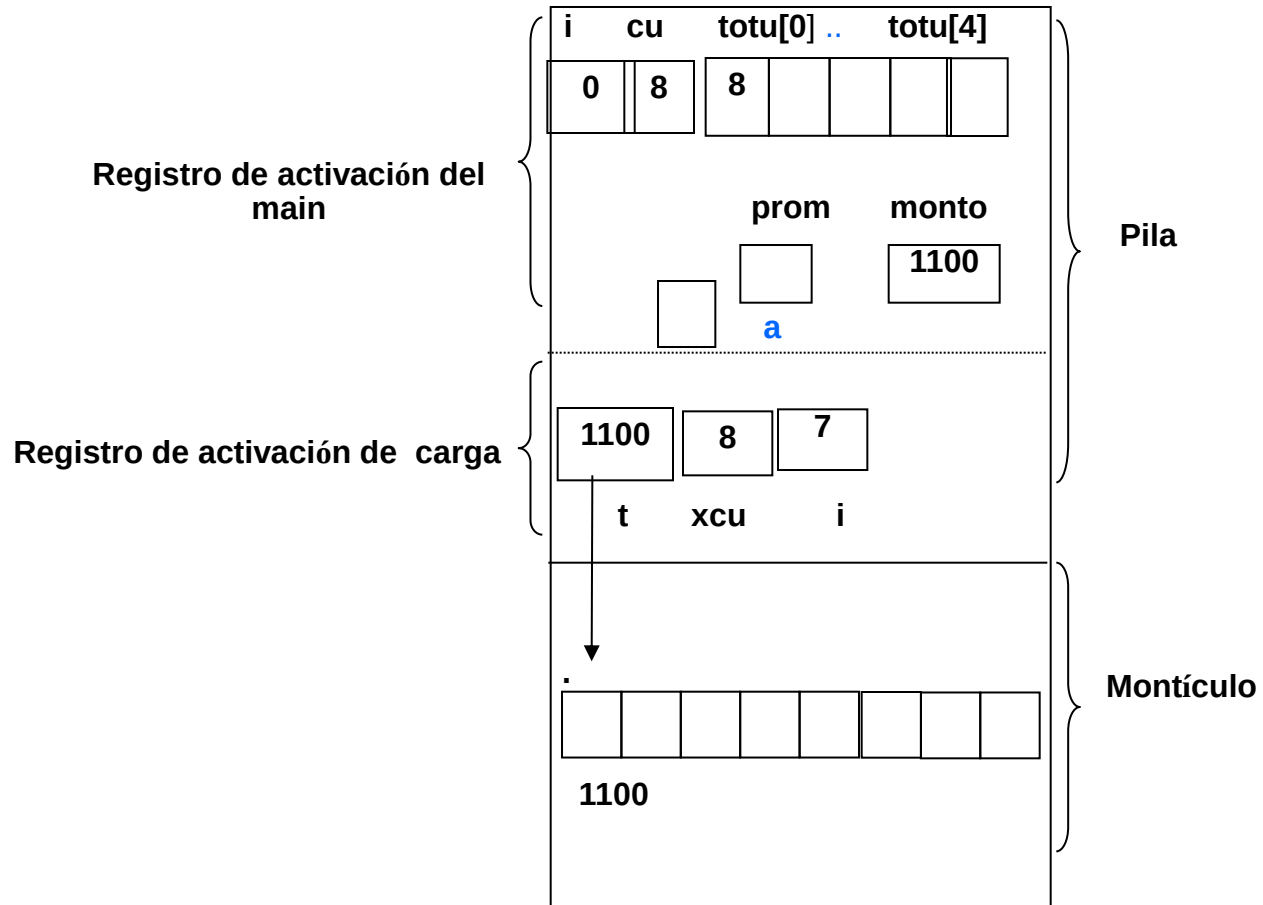
```
prom. %4d",cantidad(monto, cu, prom));
```

```
free(monto); // libera espacio asignado
```

```
}
```

```
maximo(totu, 5); }
```

Mapa de Memoria



Ejemplo

Se ingresa el registro y la nota obtenida (valor entero entre 1 y 10) por los alumnos que rindieron en la última mesa de examen de la materia Programación Procedural. Escribir un programa en C que permita:

- 1) Ingresar por teclado la cantidad de alumnos que rindieron el examen.
- 2) Mostrar la nota promedio y el número de registro de quienes obtuvieron en el examen una nota mayor o igual al promedio
- 3) Realizar un subprograma que usando la menor y la mayor nota obtenidas, indique la cantidad de alumnos que obtuvieron cada nota entera comprendida en ese rango.
- 4) Realizar el mapa de memoria después de ejecutar la función carga.

```
#include<malloc.h>
```

```
typedef struct  
{  
    int reg;  
    int nota;  
} alumno;
```



```
void carga( alumno *&al, int c)
{ int i;
  al=(alumno*) malloc(c *sizeof(alumno));
  for(i=0; i<c; i++)
  { printf("\n ingrese registro  y nota del alumno %d: ", i+1);
    scanf ("%d  %d", &al[i].reg,&al[i].nota);  }
}
```

```
void muestra( alumno *al, int c)
{ int i;
  printf("\n datos de los %d alumnos que rindieron : ", c);
  for(i=0; i<c; i++)
    printf ("\n %5d  %5d", al[i].reg, al[i].nota);
}
```

```
int main()
{ alumno *a;  int ca;
  printf("\n Ingrese la cantidad de alumnos que rindieron ");
  scanf( "%d",&ca);
  carga( a, ca );
  muestra(a, ca);
  :
}
```

Cadenas dinámicas en C

```
#include <malloc.h>
```

```
#include <string.h>
```

```
int main(void)
{
    char nom[20], *nombre;
    int t;
    printf("\n ingrese un nombre: ");
    gets(nom);
    printf("\n ingrese tamaño de nombre: ");
    scanf("%d", &t); //tamaño en forma manual
    // tamaño en forma automática
    //t = strlen(nom);
    nombre=(char *)malloc(sizeof(char) * t);
    printf("\n ingrese un nombre");
    fflush(stdin);
    gets(nombre);
    printf("\n **** nombres leidos ****\n");
    puts(nom);
    puts(nombre);
    free(nombre);
    getch();
}
```

Pila

R. main

nom

Ana Laura \0

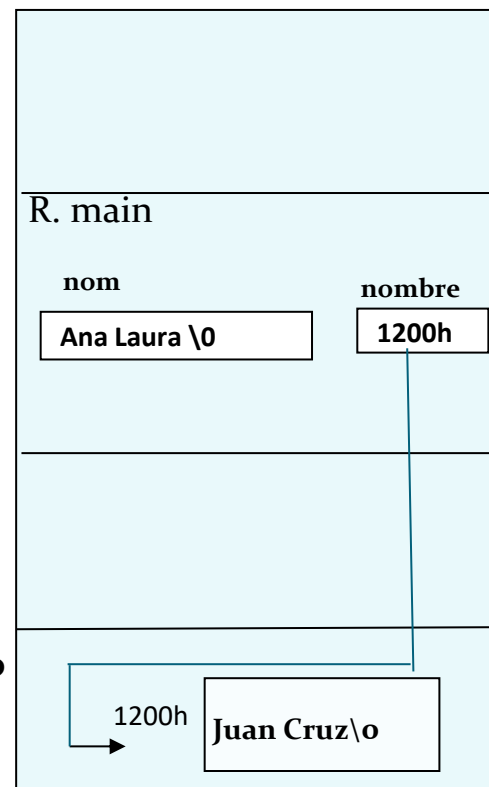
nombre

1200h

Montículo

1200h

Juan Cruz\0



Arreglos de cadenas dinámicas

EJEMPLO 4: Almacenar en memoria los nombres de 21 partidos políticos que se presentan en las próximas elecciones.(suponiendo menos de 30 caracteres)

	0	1	2	3																29
partido[0]	R	a	d	i	c	a	l	\0												
partido[1]	F	r	e	n	t	e	...								\0					
partido[2]	P	R	O	\0																
....																				
partido[20]	S	o	c	i	a	l	i	s	t	a	\0									

La declaración de este arreglo se puede hacer de dos maneras:

- `char partido[21][30];`
- `typedef char cadena[30];`
`cadena partido[21]`

Carga de los 21 nombres

	0	1	2	3									29
partido[0]	R	a	d	i	c	a	l	\0					
partido[1]	F	r	e	n	t	e	...					\0	
partido[2]	P	R	O	\0									
....													
partido[20]	S	o	c	i	a	l	i	s	t	a	\0		

Cadenas Estáticas

```
typedef char cadena[30];
```

```
cadena partido[ 21];
```

```
for( i=0; i <21; i++)
```

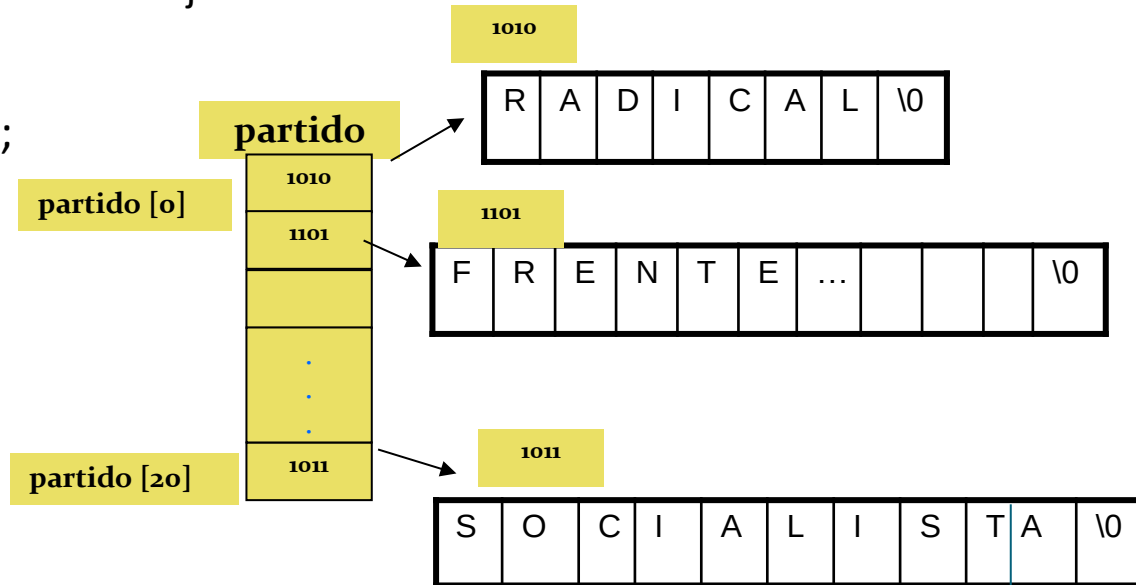
```
{
    printf("\n Ingrese un nombre");
    gets(partido[i]);
}
```

Cadenas Dinámicas

```
char * partido[21];
```

```
for( i=0; i <21; i++)
```

```
{
    printf("\n Ingrese un  nombre");
    partido[i] =(char *) malloc( l* sizeof(char));
    gets(partido[i]);
}
```



EJEMPLO: Leer y escribir, un conjunto de cadenas de caracteres de distinta longitud.

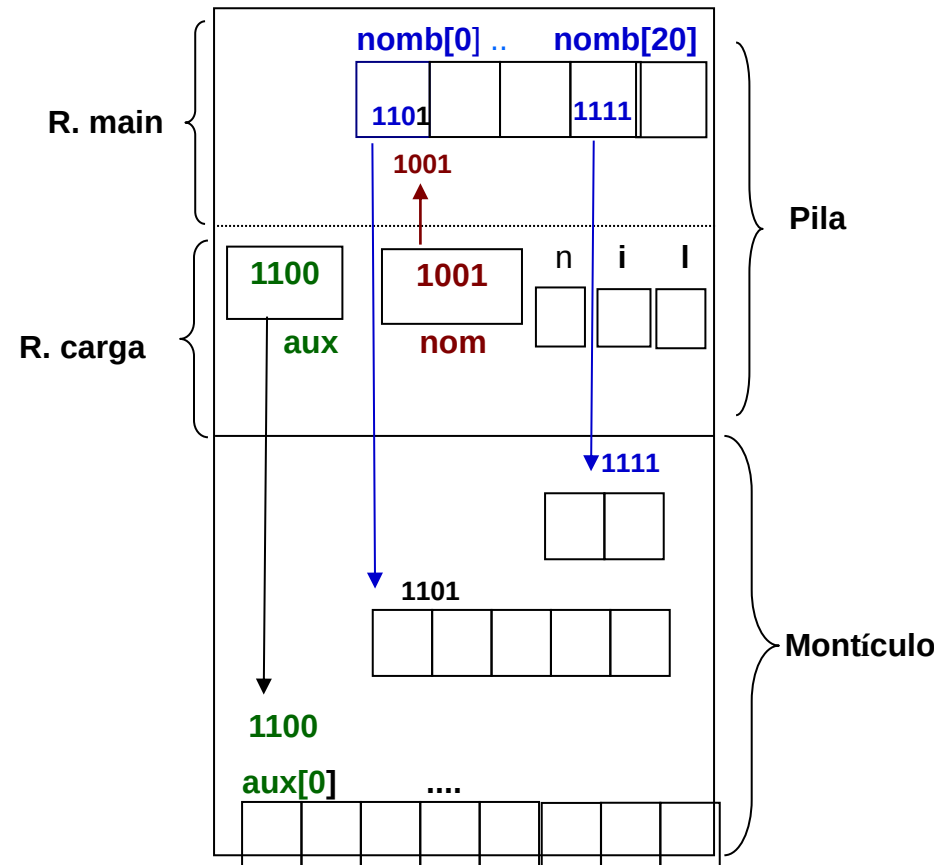
```
void carga(char *nom[], int n)
{
    char *aux;
    int i, l;
    aux=(char*) malloc(30*sizeof(char));
    /* solicita memoria para la variable aux */
    puts(" Ingrese Nombres:\n");

    for(i=0; i<n; i++)
    {
        printf("\n Partido %d: ", i+1);    gets(aux);
        l = strlen(aux)+1;
        nom[i] =(char *) malloc( l* sizeof(char));
        strcpy(nom[i],aux);
    }
    free (aux);
}
```

```
void muestra(char *nom[], int cfil)
{
    for( int i=0; i<cfil; i++)
        puts(nom[i]);
}
```

```
void main(void)
{
    char *nomb[21];

    carga(nomb,21);
    puts("\n Partidos Ingresados: ");
    muestra(nomb,21);
}
```



Ejemplo

Suponga que cuenta con los nombres y edad de 50 empleados de una empresa y desea:

a - Mostrar el nombre del empleado de mayor edad (suponer único).

b - Dado un carácter, decir la cantidad de veces que aparece en los nombres de los empleados.

Completar el siguiente programa con las funciones que le permitan resolver la problemática planteada.

```
#include <stdio.h>
#include <string.h>
const int n=50;
typedef struct
{ char *nom;
  int edad;
} empleados;
.... carga (.....)

char* maximo(empleados x[n])
{ char *p;
  for(i=0; i<n; i++)
    if (x[i].edad >max)
      { p=x[i].nom;
        max=x[i].edad;
      };
  return p;
}

int mostrar (empleados x[], char r)
{ int i,j,l,c=0;
  for(i=0; i<n; i++)
    { l=strlen( x[i].nom);
      for(j=0;j<l ; j++)
        if (x[i].nom[j]== 'r') c++ ;
    }
  return(c);
}
```

```
int main()
{ char e;
  empleados a[n];
  carga(a);
  printf("\ mayor edad %s ", maximo(a));
  printf(" \n ingrese un caracter %c",e);
  e=getchar();
  mostrar(a, e);
  muestra(a);
  libera(a);
  printf(" \n despues de liberar");
  muestra(a);
  getch();
}
```

```
void libera (empleados x[])
```

```
{  
    int i;  
    for(i=0; i<n; i++)  
        { free( x[i].nom);  
          // x[i].nom= NULL;  
        } ¿es necesario colocar la sentencia x[i].nom= NULL ?  
}
```

```
void muestra (empleados x[])
```

```
{  
    int i;  
    for(i=0; i<n; i++)  
        puts( x[i].nom);  
}
```

```
int main()
```

```
{ char e;  
    empleados a[n];  
    carga(a);  
    printf("\ mayor edad %s ", maximo(a));  
    printf("\n ingrese un caracter %c",e);  
    e=getchar();  
    mostrar(a, e);  
    muestra(a);  
    libera(a);  
    printf("\n despues de liberar");  
    muestra(a);  
    getch();  
}
```